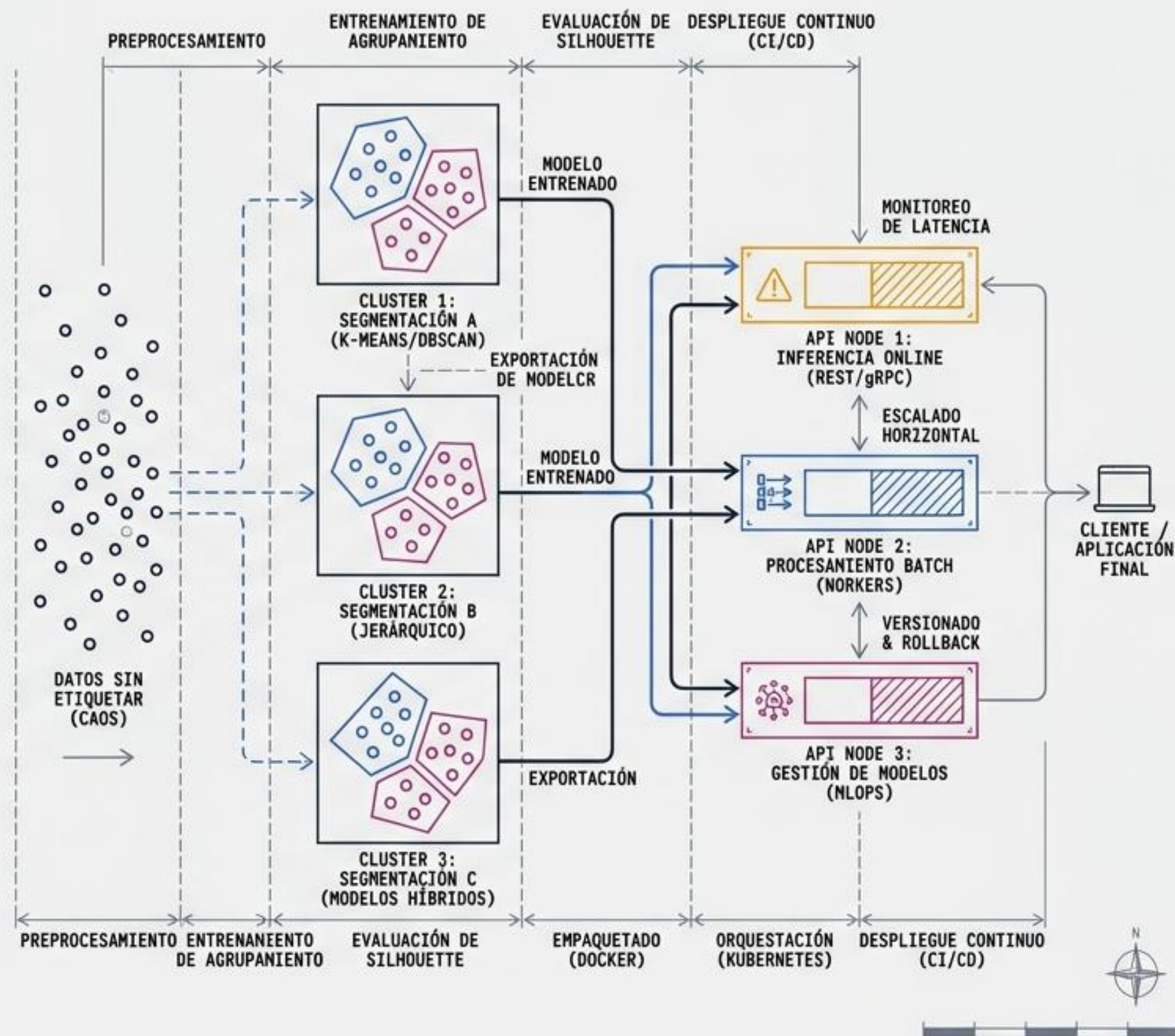


Modelos de Agrupamiento y Despliegue de Soluciones ML

Del caos de datos sin etiquetar a la arquitectura de microservicios



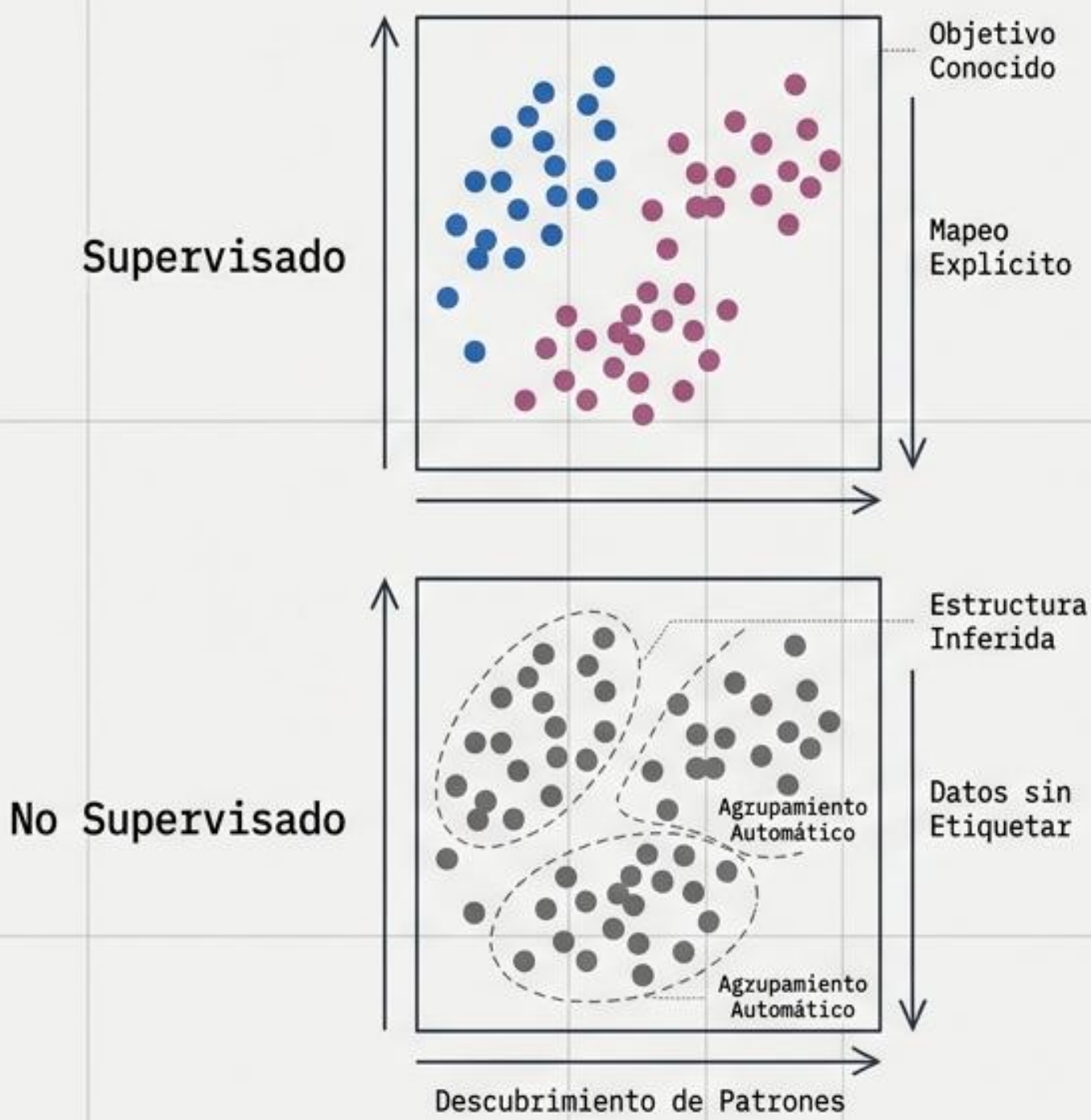
Asignatura: Ingeniería del Conocimiento (IS056B)

Programa: Ingeniería de Sistemas – UNCP

Docente: Msc. Jaime Antonio Huaytalla Pariona

Nivel: Pregrado, intermedio-avanzado | Semestre: 2026-II

Descubriendo la Estructura Oculta



Métricas de Distancia Clave

Métrica	Fórmula	Uso
Euclidiana	$d(x, y) = \sqrt{\sum (x_i - y_i)^2}$	Datos numéricos, clusters esféricos
Manhattan (L1)	$d(x, y) = \sum x_i - y_i $	Alta dimensionalidad, cuadrículas
Coseno	$sim(x, y) = \frac{x \cdot y}{ x y }$	Texto/TF-IDF, embeddings
Minkowski	$d(x, y) = \left(\sum x_i - y_i ^p \right)^{\frac{1}{p}}$	Generalización de L1 y L2

⚠ PRECAUCIÓN CRÍTICA:

Antes de aplicar métricas basadas en distancia, es obligatorio estandarizar las variables continuas (StandardScaler). Sin esto, variables con mayor magnitud dominarán por completo el cálculo geométrico.

K-Means: Matemáticas

Objetivo: Minimizar la Inercia (WCSS)

$$J(\mathbf{c}, \boldsymbol{\mu}) = \sum \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_{c(i)}\|^2$$

- $\mathbf{x}^{(i)}$ = observación
- $c^{(i)}$ = cluster asignado
- $\boldsymbol{\mu}_j$ = centroide del cluster

Algoritmo Iterativo (Lloyd)

Descenso por coordenadas

1. ASIGNACIÓN

$$c^{(i)} := \arg \min_j \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$$

2. ACTUALIZACIÓN

$$\boldsymbol{\mu}_j := \frac{1}{|C_j|} \sum \mathbf{x}_i$$

Implementación Python

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)
kmeans = KMeans(n_clusters=3, max_iter=300)
labels = kmeans.fit_predict(X_scaled)

print(f'Inercia: {kmeans.inertia_:.2f}')
```

Estrategias de Inicialización y Escalabilidad

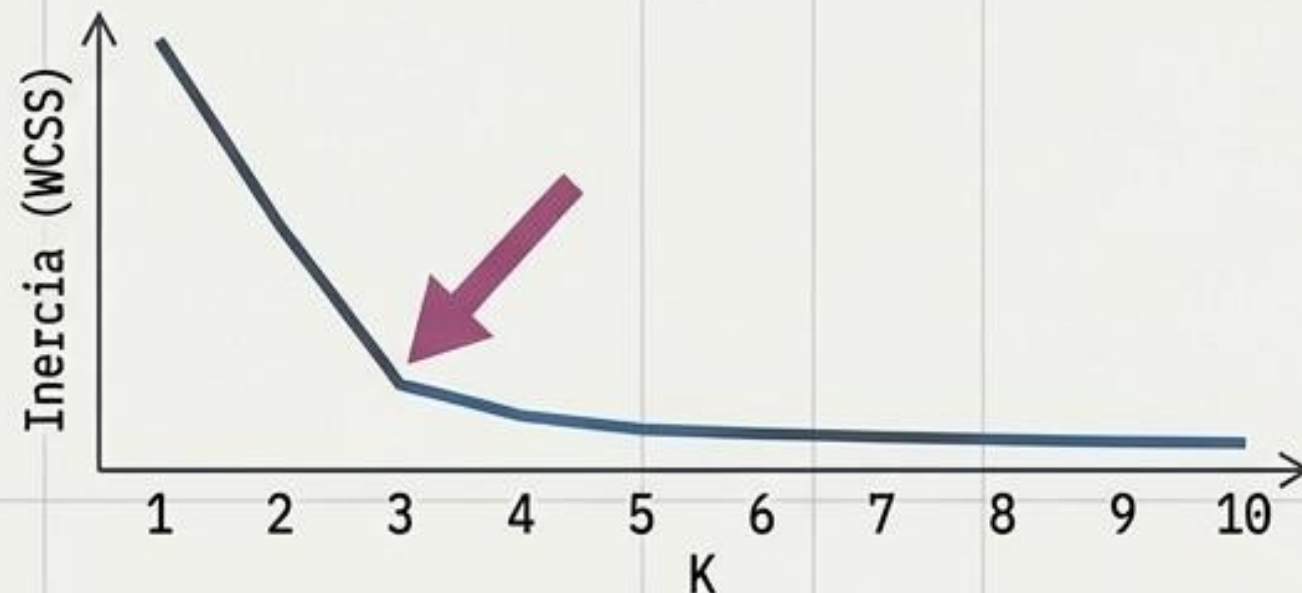
K-Means++

Selecciona el primer centroide al azar, y los siguientes con probabilidad proporcional a la distancia al cuadrado $D(x_i)^2$. Evita caer en mínimos locales subóptimos.

Mini-Batch K-Means

Actualiza centroides usando mini-lotes aleatorios en lugar de todo el dataset. Reduce la complejidad temporal a $O(n)$, ideal para más de 10,000 muestras.

Método del Codo (Elbow Method)



Busca el punto de inflexión.
Limitación: Alta subjetividad.

Análisis de Silueta

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

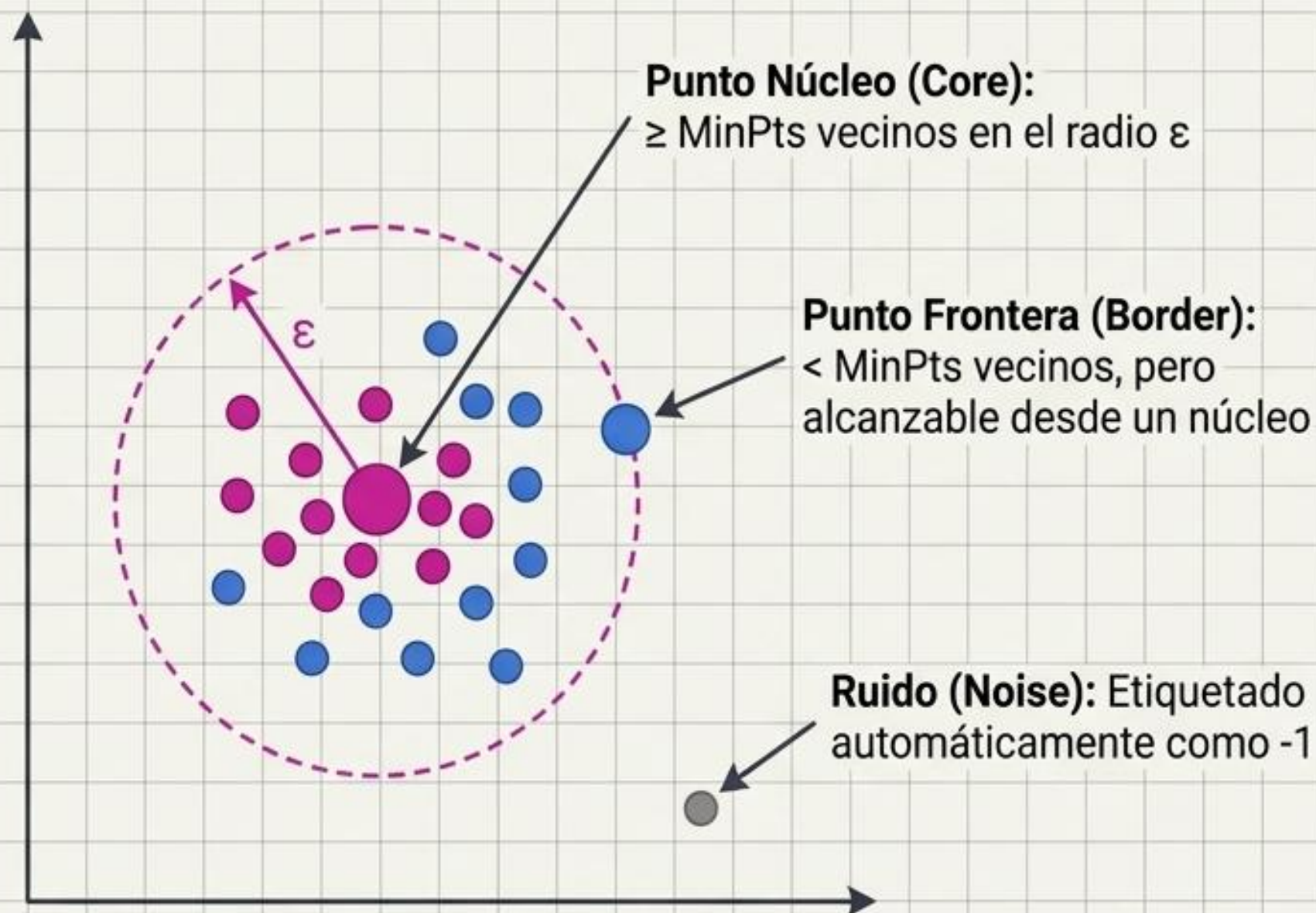
$a(i)$ = distancia media intra-cluster

$b(i)$ = distancia media al cluster vecino

Rango: $[-1, 1]$. Cercano a 1 indica excelente asignación.

```
silhouette_score(X_scaled, labels)
```

DBSCAN: Agrupamiento Espacial Basado en Densidad



Ventaja: No asume clusters esféricos.

Selección del parámetro ϵ

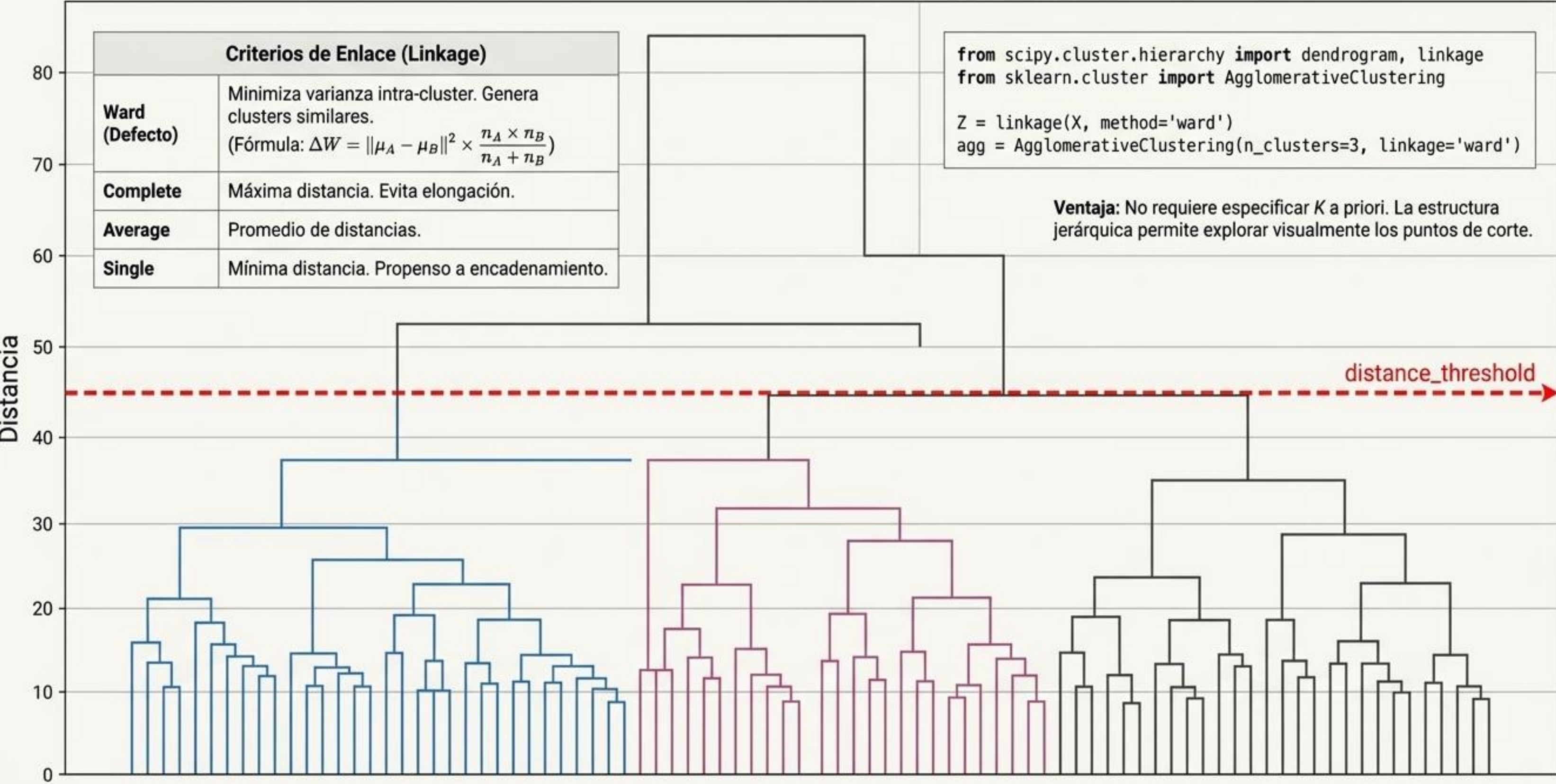


```
from sklearn.cluster import DBSCAN
```

```
dbscan = DBSCAN(eps=0.5, min_samples=5)  
labels = dbscan.fit_predict(X_scaled)
```

```
n_noise = list(labels).count(-1)
```

Hierarchical Clustering & Dendrogram



GMM y el Algoritmo EM (Expectation-Maximization)

Generative model

$$p(\mathbf{x}) = \sum \pi_k \cdot N(\mathbf{x} \mid \mu_k, \Sigma_k)$$

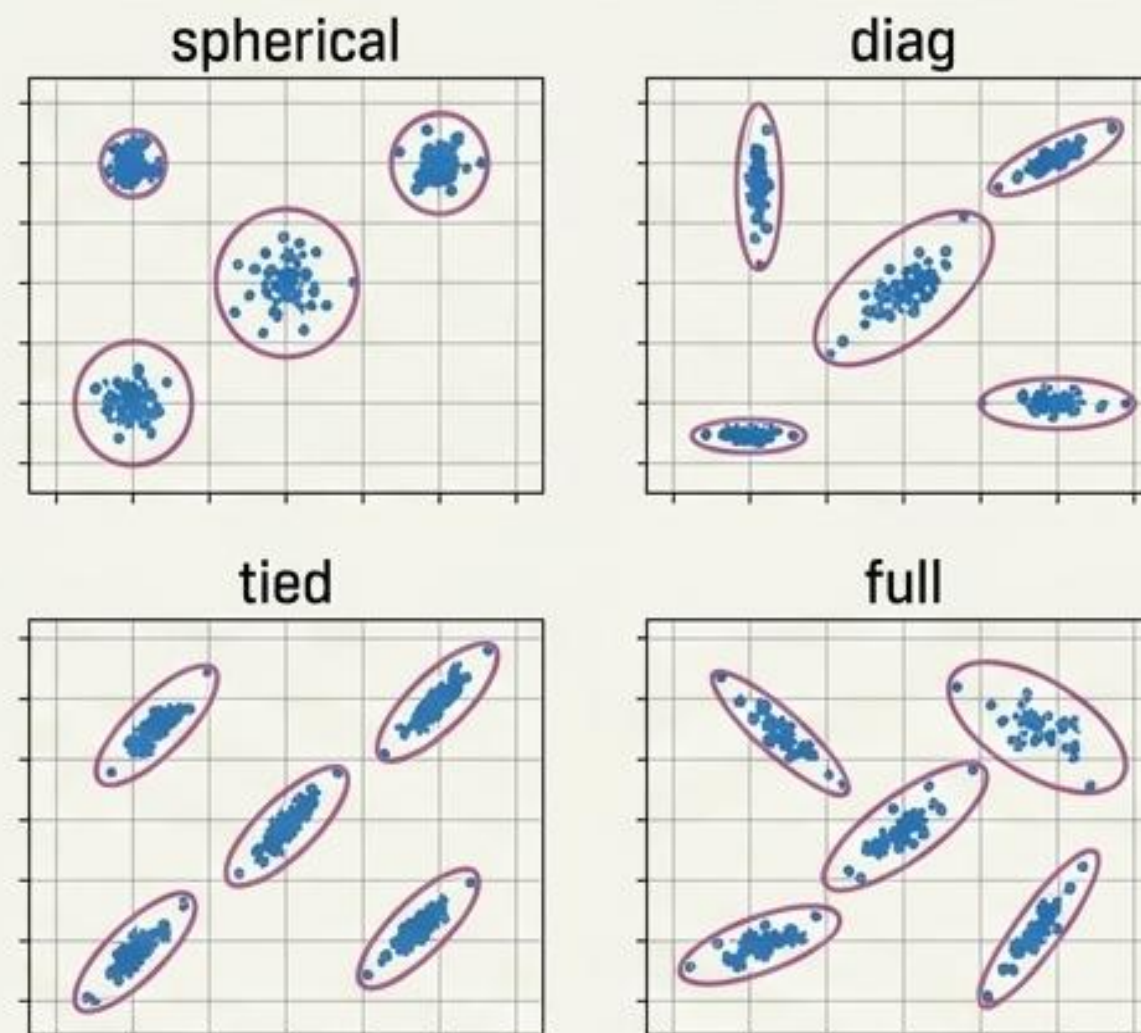
E-STEP (Expectación)

Calcula responsabilidades
(asignación probabilística suave).
 $\gamma(i,k)$ = prob. de que x_i pertenezca
al componente k

M-STEP (Maximización)

Actualiza los parámetros basándose
en las responsabilidades:
 μ_k (medias), Σ_k (covarianzas),
 π_k (pesos)

Estructuras de Covarianza (Σ)



Selección de Componentes: Usar BIC (Bayesian Information Criterion) para penalizar la complejidad.

```
gmm = GaussianMixture(n_components=3, covariance_type='full')  
gmm.fit(X)  
probabilidades = gmm.predict_proba(X)
```

Evaluación: ¿Qué tan bueno es el Agrupamiento?

Métricas Internas (Evaluación Empírica, sin etiquetas)

Silhouette Score

Evalúa la forma y separación espacial.

Rango: $[-1, 1]$. Valores más cercanos a 1 indican clusters perfectamente separados y densos.

Calinski-Harabasz (CH)

Ratio entre la dispersión inter-cluster y la dispersión intra-cluster.

Fórmula:
$$\frac{\text{Tr}(B_k)}{\text{Tr}(W_k)} \times \frac{(n - k)}{(k - 1)}$$

Rango: $[0, \infty)$. Mayor valor es mejor.

Davies-Bouldin (DB)

Mide el ratio de compacidad promedio frente a la separación de los centroides.

Rango: $[0, \infty)$. MENOR valor es mejor (más compacto y separado).

Métricas Externas (Benchmarks con Etiquetas Reales)

Adjusted Rand Index (ARI)

Mide la similitud de asignaciones ignorando permutaciones, corregido por el azar.

Fórmula:
$$\frac{(RI - E[RI])}{(\max(RI) - E[RI])}$$

Rango: $[-0.5, 1]$. Valores altos = alta concordancia con ground truth.

Adjusted Mutual Information (AMI)

Evalúa la cantidad de información compartida entre predicciones y realidad, corregida por azar. Rango: $[0, 1]$.

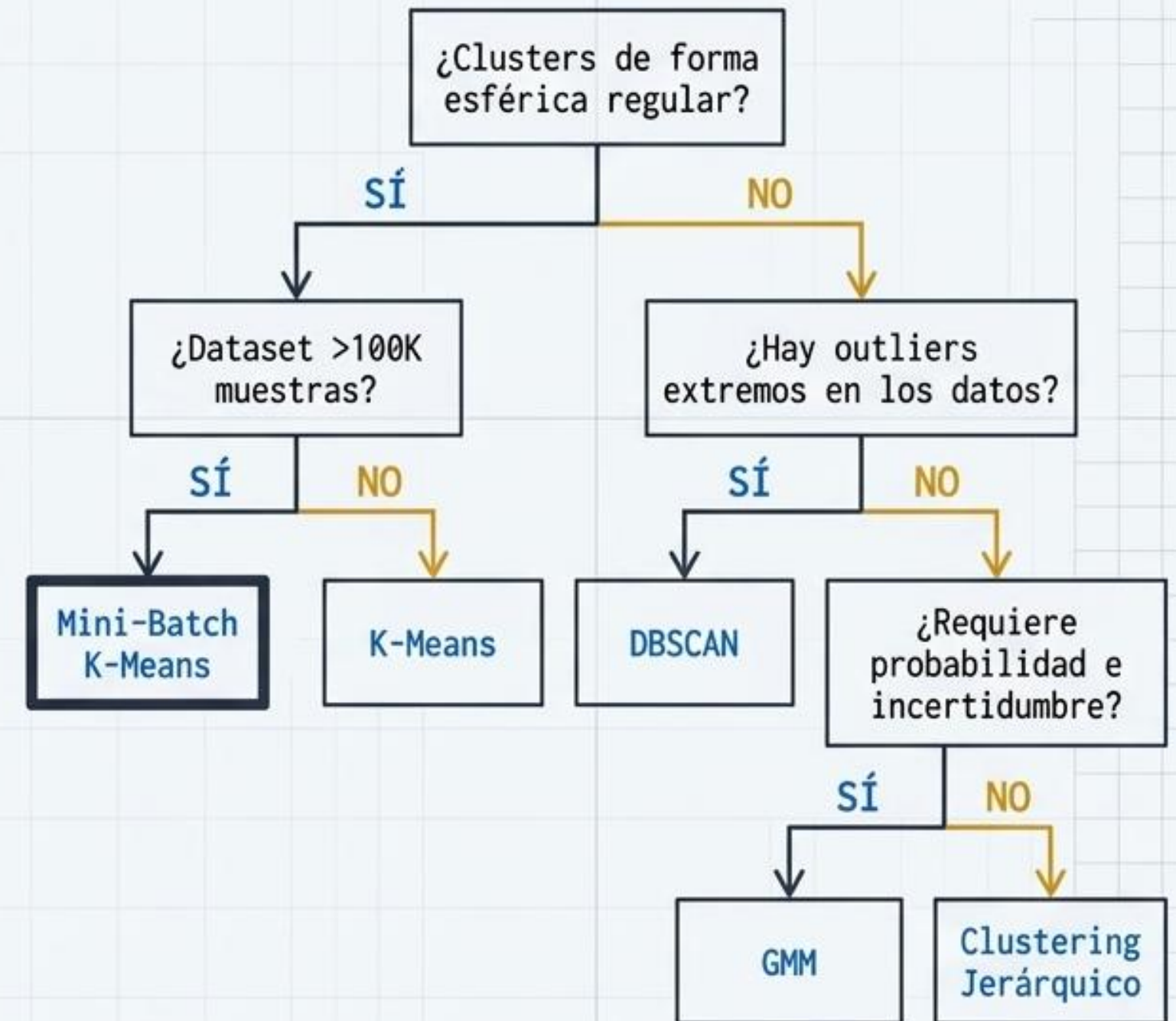
V-Measure

Media armónica entre Homogeneidad (cada cluster tiene una sola clase) y Completitud (toda la clase está en un solo cluster).

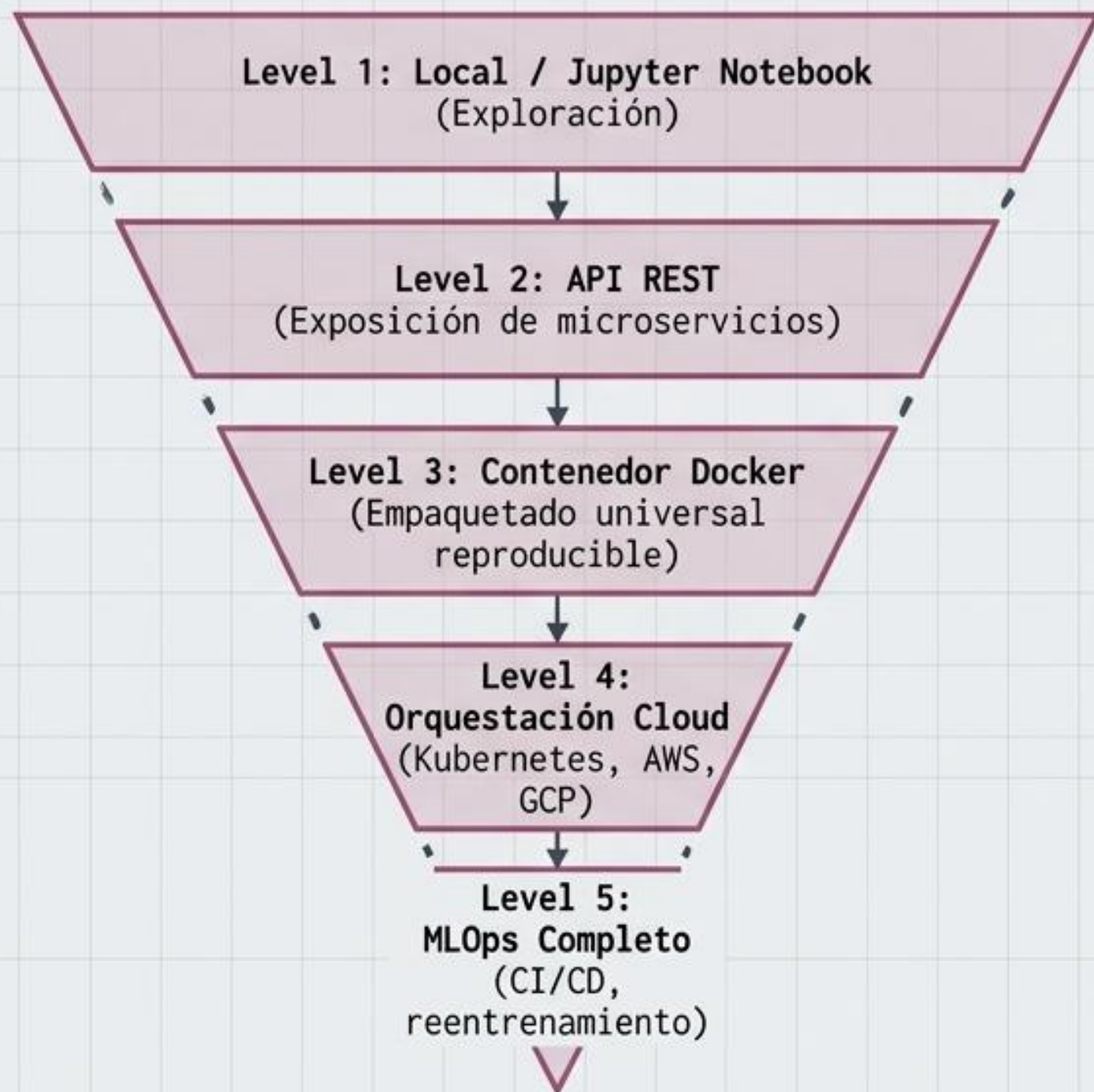
Matriz de Diagnóstico de Algoritmos

Criterio	K-Means	DBSCAN	Jerárquico	GMM
Requiere K	Sí	No	Opcional	Sí
Forma de clusters	Esférica	Arbitraria	Variable	Elipsoidal
Manejo de Outliers	Sensible	Etiqueta -1	Sensible	Probabilidad baja
Complejidad Temporal	$O(n \cdot k)$	$O(n \log n)$	$O(n^3)$	$O(n \cdot k \cdot d^2)$

Árbol de Decisión del Arquitecto de Datos



El Abismo del Despliegue ML



Persistencia del Conocimiento (Serialización)

joblib : Eficiente para arrays NumPy.	Seguridad: Riesgo alto.
pickle : Nativo de Python.	Seguridad: Riesgo alto.
ONNX / skops : Validación estricta, independiente.	Seguridad: Excelente.

```
from joblib import dump, load

# Preservar el modelo y el preprocesador
dump(model, 'artifacts/modelo_clustering.joblib')
dump(scaler, 'artifacts/scaler.joblib')
```

⚠ RIESGO DE SEGURIDAD: Nunca cargue archivos .pkl o .joblib de fuentes no confiables. Estos formatos pueden ejecutar código arbitrario silenciosamente durante la deserialización.

Arquitectura de una API REST para Modelos Serializados

Estructura del Proyecto

```
ml-api/  
├── app/  
│   └── main.py          # Servidor FastAPI  
├── artifacts/  
│   ├── model.joblib     # Modelo congelado  
│   └── scaler.joblib    # Preprocesador  
├── requirements.txt     # Dependencias  
└── Dockerfile           # Receta de entorno
```

El Viaje del Payload (Flujo de Arquitectura Inferencia)



Implementación del Endpoint con FastAPI

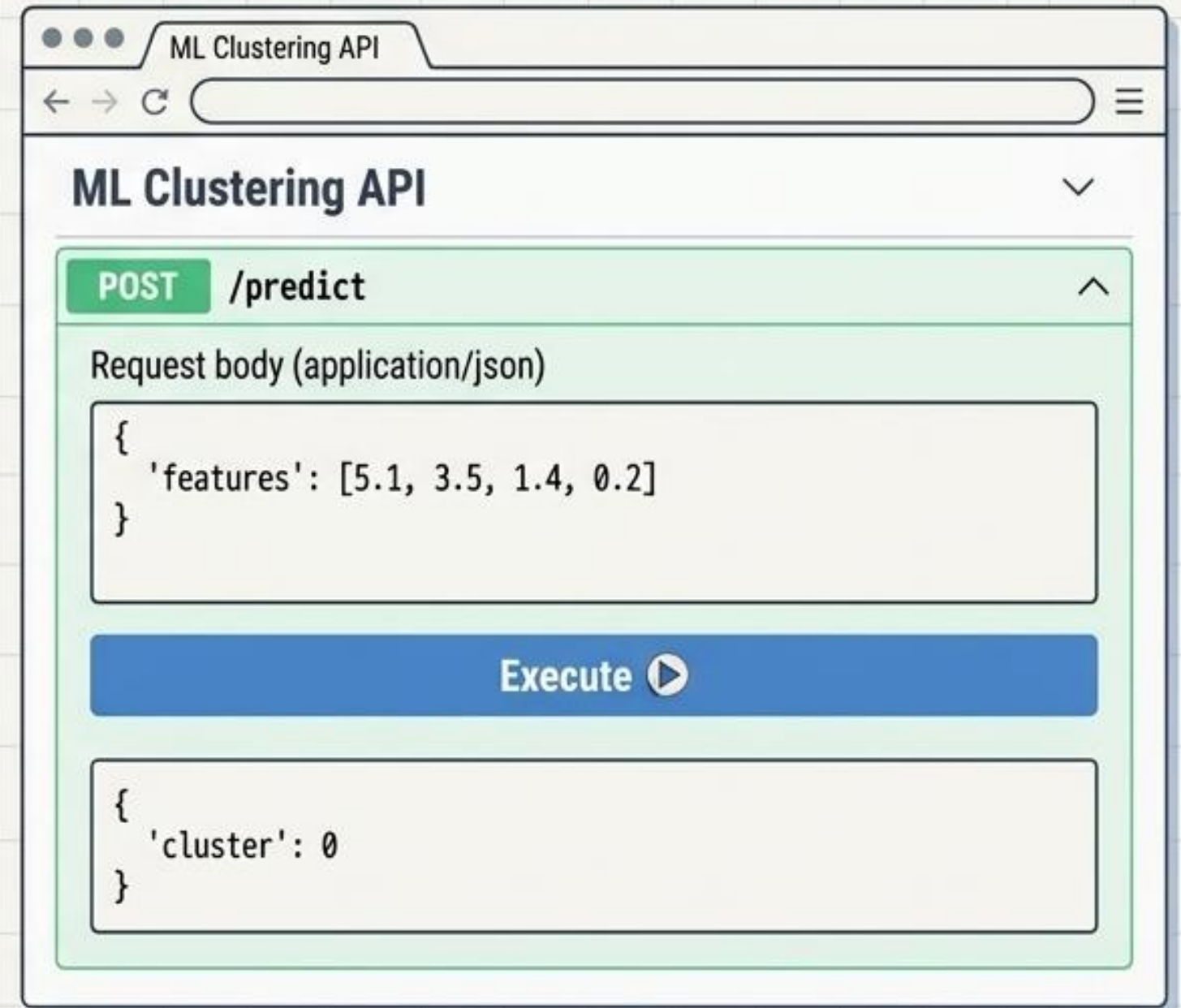
```
from fastapi import FastAPI
from pydantic import BaseModel
from joblib import load
import numpy as np

app = FastAPI(title='ML Clustering API')
model = load('artifacts/model.joblib')
scaler = load('artifacts/scaler.joblib')

class PredictionRequest(BaseModel):
    features: list[float]

@app.post('/predict')
def predict(request: PredictionRequest):
    X = np.array(request.features).reshape(1, -1)
    X_scaled = scaler.transform(X)
    cluster = int(model.predict(X_scaled)[0])
    return {'cluster': cluster}
```

Documentación Automática Interactiva (Swagger UI)



Comando de ejecución local: `fastapi dev app/main.py`

Contenerización: Empaquetado Universal con Docker

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ app/
COPY artifacts/ artifacts/
EXPOSE 8000
CMD ['fastapi', 'run', 'app/main.py', '--host',
    '0.0.0.0', '--port', '8000']
```

Imagen base ligera de Python para minimizar el tamaño final.

Copiar el txt y ejecutar pip ANTES del código fuente es el secreto para maximizar la eficiencia del caché de capas de Docker.

Inyección de los modelos congelados y matrices de preprocesamiento.

Comandos de Despliegue

```
$ docker build -t ml-clustering-api .
$ docker run -p 8000:8000 ml-clustering-api
```

Prototipado Rápido de Interfaces (Streamlit)

Construcción de interfaces web interactivas puramente en Python para validación técnica.

```
import streamlit as st
f1 = st.slider('Feature 1', 0.0, 10.0, 5.0)
if st.button('Predecir'):
    cluster = model.predict(scaler.transform([f1])[0])
    st.success(f'Cluster: {cluster}')
```



Trazabilidad y Madurez MLOps (MLflow)

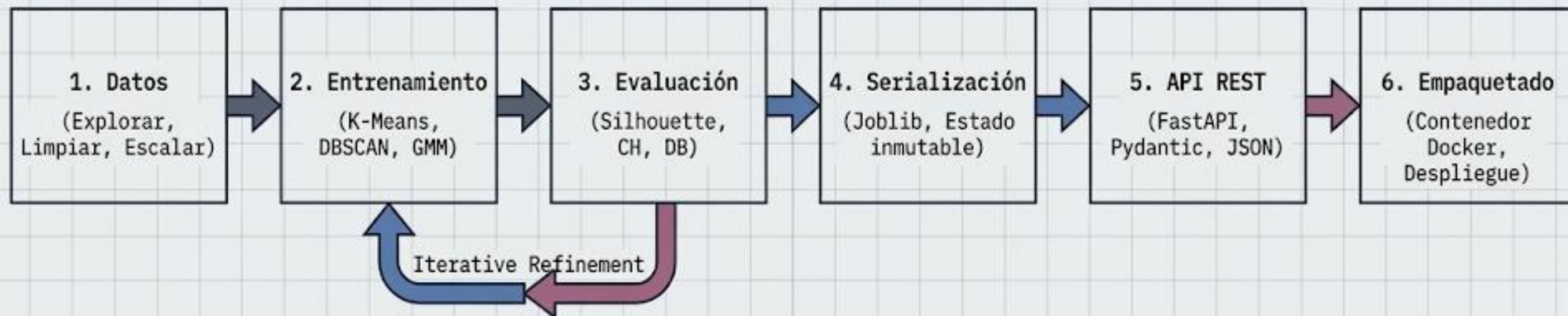
Registro sistemático de hiperparámetros, métricas y versionado de modelos durante la experimentación.

```
import mlflow
with mlflow.start_run():
    mlflow.log_param('n_clusters', 3)
    mlflow.log_metric('silhouette', sil_score)
    mlflow.sklearn.log_model(kmeans, 'model')
```

Componentes clave:

- Tracking (Métricas)
- Models (Empaquetado)
- Registry (Promoción de versiones)

El Pipeline Completo MLOps



Bibliografía Fundamental para Exploración Autónoma

Fundamentos Matemáticos

- Bishop (2006): Pattern Recognition and ML.
- Hastie et al. (2009): Elements of Statistical Learning.

Papers Seminales

- Lloyd (1982): K-Means original.
- Ester et al. (1996): DBSCAN.
- Dempster et al. (1977): Algoritmo EM para GMM.

Ingeniería Aplicada

- Géron (2023): Hands-On ML with Scikit-Learn.
- Documentación oficial: scikit-learn, FastAPI, Docker.